FINAL PROJECT REPORT

STEAMED BUN BUSINESS MANAGEMENT SYSTEM
for Family-Operated Steamed Bun Business

Course: 977-210 Object-Oriented Programming

Prepared By:
Name: Patthariyakorn Laksanakun
Student ID: 6930613015

Program: Digital Engineering
Faculty: Faculty of Computing
Prince of Songkla University, Phuket Campus
Academic Year: 2026

BunItem.java

```java
public interface BunItem {
    String getProductId();
    String getName();
    double getPrice();
    int getQuantity();
    String getCategory();
    double calculateStockValue();
}
```

AbstractBun.java

```
public abstract class AbstractBun implements BunItem {
    // Encapsulation: Private attributes
    private String productId;
    private String name;
    private double price;
    private int quantity;

    public AbstractBun(String productId, String name, double price, int
quantity) {
        this.productId = productId;
        this.name = name;
        this.price = price;
        this.quantity = quantity;
    }

    // Getters and Setters
    @Override
    public String getProductId() { return productId; }
    public void setProductId(String productId) { this.productId =
productId; }

    @Override
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    @Override
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }

    @Override
    public int getQuantity() { return quantity; }
    public void setQuantity(int quantity) { this.quantity = quantity; }

    // Abstract method from interface
    @Override
    public abstract String getCategory();

    // Final method: Cannot be overridden by subclasses
    @Override
    public final double calculateStockValue() {
        return price * quantity;
```

```
    }

    @Override
    public String toString() {
        return String.format("ID: %s | Name: %s | Price: %.2f | Qty: %d
| Stock Value: %.2f",
                productId, name, price, quantity,
calculateStockValue());
    }
}
```

SavoryBun.java

```
public class SavoryBun extends AbstractBun {
    // Unique attribute for SavoryBun
    private String spiceLevel;

    public SavoryBun(String productId, String name, double price, int
quantity, String spiceLevel) {
        super(productId, name, price, quantity);
        this.spiceLevel = spiceLevel;
    }

    // Unique methods
    public String getSpiceLevel() { return spiceLevel; }
    public void setSpiceLevel(String spiceLevel) { this.spiceLevel =
spiceLevel; }

    @Override
    public String getCategory() {
        return "Savory Bun";
    }

    @Override
    public String toString() {
        return super.toString() + " | Category: " + getCategory() + " |
Spice Level: " + spiceLevel;
    }
}
```

SweetBun.java

```
public class SweetBun extends AbstractBun {
    // Unique attribute for SweetBun
    private String sweetnessLevel;

    public SweetBun(String productId, String name, double price, int
quantity, String sweetnessLevel) {
        super(productId, name, price, quantity);
        this.sweetnessLevel = sweetnessLevel;
    }

    // Unique methods
    public String getSweetnessLevel() { return sweetnessLevel; }
    public void setSweetnessLevel(String sweetnessLevel) {
this.sweetnessLevel = sweetnessLevel; }

    @Override
    public String getCategory() {
        return "Sweet Bun";
    }

    @Override
    public String toString() {
        return super.toString() + " | Category: " + getCategory() + " |
Sweetness Level: " + sweetnessLevel;
    }
}
```

SteamedBunApp.java

```
import javax.swing.*;
import java.awt.*;
import java.util.ArrayList;

public class SteamedBunApp extends JFrame {
    // Shared Collection using Subtype Polymorphism
    private ArrayList<BunItem> bunList = new ArrayList<>();

    // GUI Components for Navigation (CardLayout)
    private CardLayout cardLayout = new CardLayout();
    private JPanel mainPanel = new JPanel(cardLayout);

    // Data Entry Page Components
    private JTextField txtId, txtName, txtPrice, txtQuantity,
txtSpecific;
    private JComboBox<String> cbType;
    private JLabel lblSpecific;
    private JTextArea txtAreaDisplay;

    // Search Page Components
    private JComboBox<String> cbSearchCriteria;
    private JTextField txtSearchValue;
    private JTextArea txtAreaSearchResults;

    public SteamedBunApp() {
        setTitle("Steamed Bun Business Management System");
        setSize(700, 500);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        // Build Pages
        mainPanel.add(createLandingPage(), "Landing");
        mainPanel.add(createDataEntryPage(), "DataEntry");
        mainPanel.add(createSearchPage(), "Search");

        add(mainPanel);
    }

    // --- 1. LANDING PAGE ---
    private JPanel createLandingPage() {
        JPanel panel = new JPanel(new GridLayout(4, 1, 10, 10));
```

```
        panel.setBorder(BorderFactory.createEmptyBorder(30, 50, 30,
50));

        JLabel lblTitle = new JLabel("Steamed Bun Management System",
SwingConstants.CENTER);
        lblTitle.setFont(new Font("Tahoma", Font.BOLD, 18));

        JButton btnDataEntry = new JButton("Go to Data Entry Page");
        JButton btnSearch = new JButton("Go to Search Page");
        JButton btnExit = new JButton("Exit");

        btnDataEntry.addActionListener(e -> cardLayout.show(mainPanel,
"DataEntry"));
        btnSearch.addActionListener(e -> cardLayout.show(mainPanel,
"Search"));
        btnExit.addActionListener(e -> System.exit(0));

        panel.add(lblTitle);
        panel.add(btnDataEntry);
        panel.add(btnSearch);
        panel.add(btnExit);

        return panel;
    }

    // --- 2. DATA ENTRY PAGE ---
    private JPanel createDataEntryPage() {
        JPanel panel = new JPanel(new BorderLayout(10, 10));
        panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10,
10));

        // Form Fields Panel
        JPanel formPanel = new JPanel(new GridLayout(6, 2, 5, 5));

        txtId = new JTextField();
        txtName = new JTextField();
        txtPrice = new JTextField();
        txtQuantity = new JTextField();
        txtSpecific = new JTextField();
        cbType = new JComboBox<>(new String[]{"Savory Bun", "Sweet
Bun"});
        lblSpecific = new JLabel("Spice Level:");
```

```
        cbType.addActionListener(e -> {
            if (cbType.getSelectedItem().equals("Savory Bun")) {
                lblSpecific.setText("Spice Level:");
            } else {
                lblSpecific.setText("Sweetness Level:");
            }
        });

        formPanel.add(new JLabel("Product Type:"));
        formPanel.add(cbType);
        formPanel.add(new JLabel("Product ID:"));
        formPanel.add(txtId);
        formPanel.add(new JLabel("Product Name:"));
        formPanel.add(txtName);
        formPanel.add(new JLabel("Price (THB):"));
        formPanel.add(txtPrice);
        formPanel.add(new JLabel("Quantity:"));
        formPanel.add(txtQuantity);
        formPanel.add(lblSpecific);
        formPanel.add(txtSpecific);

        // Buttons Panel
        JPanel btnPanel = new JPanel(new FlowLayout());
        JButton btnAdd = new JButton("Add Product");
        JButton btnClear = new JButton("Clear");
        JButton btnBack = new JButton("Back to Main");

        btnAdd.addActionListener(e -> addProduct());
        btnClear.addActionListener(e -> clearDataEntryFields());
        btnBack.addActionListener(e -> cardLayout.show(mainPanel,
"Landing"));

        btnPanel.add(btnAdd);
        btnPanel.add(btnClear);
        btnPanel.add(btnBack);

        // Display Area
        txtAreaDisplay = new JTextArea(10, 50);
        txtAreaDisplay.setEditable(false);
        JScrollPane scrollPane = new JScrollPane(txtAreaDisplay);

        JPanel topContainer = new JPanel(new BorderLayout());
        topContainer.add(formPanel, BorderLayout.CENTER);
```

```
        topContainer.add(btnPanel, BorderLayout.SOUTH);

        panel.add(topContainer, BorderLayout.NORTH);
        panel.add(scrollPane, BorderLayout.CENTER);

        return panel;
    }

    // Data Entry Logic & Exception Handling
    private void addProduct() {
        try {
            String id = txtId.getText().trim();
            String name = txtName.getText().trim();
            String specific = txtSpecific.getText().trim();

            if (id.isEmpty() || name.isEmpty() || specific.isEmpty()) {
                throw new IllegalArgumentException("All fields must be
filled out!");
            }

            double price =
Double.parseDouble(txtPrice.getText().trim());
            int quantity =
Integer.parseInt(txtQuantity.getText().trim());

            if (price < 0 || quantity < 0) {
                throw new IllegalArgumentException("Price and Quantity
must be non-negative!");
            }

            BunItem newBun;
            if (cbType.getSelectedItem().equals("Savory Bun")) {
                newBun = new SavoryBun(id, name, price, quantity,
specific);
            } else {
                newBun = new SweetBun(id, name, price, quantity,
specific);
            }

            bunList.add(newBun);
            JOptionPane.showMessageDialog(this, "Product added
successfully!");
            clearDataEntryFields();
```

```
            refreshDataEntryDisplay();

        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Please enter valid
numeric values for Price and Quantity.", "Input Error",
JOptionPane.ERROR_MESSAGE);
        } catch (IllegalArgumentException ex) {
            JOptionPane.showMessageDialog(this, ex.getMessage(), "Input
Error", JOptionPane.WARNING_MESSAGE);
        }
    }

    private void clearDataEntryFields() {
        txtId.setText("");
        txtName.setText("");
        txtPrice.setText("");
        txtQuantity.setText("");
        txtSpecific.setText("");
    }

    private void refreshDataEntryDisplay() {
        StringBuilder sb = new StringBuilder();
        for (BunItem item : bunList) {
            sb.append(item.toString()).append("\n");
        }
        txtAreaDisplay.setText(sb.toString());
    }

    // --- 3. SEARCH PAGE ---
    private JPanel createSearchPage() {
        JPanel panel = new JPanel(new BorderLayout(10, 10));
        panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10,
10));

        JPanel searchControlPanel = new JPanel(new FlowLayout());
        cbSearchCriteria = new JComboBox<>(new String[]{
            "Product ID", "Product Name", "Price", "Quantity", "Spice
Level", "Sweetness Level"
        });
        txtSearchValue = new JTextField(15);
        JButton btnSearch = new JButton("Search");
        JButton btnBack = new JButton("Back to Main");
```

```
        searchControlPanel.add(new JLabel("Search By:"));
        searchControlPanel.add(cbSearchCriteria);
        searchControlPanel.add(new JLabel("Value:"));
        searchControlPanel.add(txtSearchValue);
        searchControlPanel.add(btnSearch);
        searchControlPanel.add(btnBack);

        txtAreaSearchResults = new JTextArea(15, 50);
        txtAreaSearchResults.setEditable(false);
        JScrollPane scrollPane = new JScrollPane(txtAreaSearchResults);

        btnSearch.addActionListener(e -> performSearch());
        btnBack.addActionListener(e -> cardLayout.show(mainPanel,
"Landing"));

        panel.add(searchControlPanel, BorderLayout.NORTH);
        panel.add(scrollPane, BorderLayout.CENTER);

        return panel;
    }

    // Search Logic with Polymorphism Handling
    private void performSearch() {
        String criteria = (String) cbSearchCriteria.getSelectedItem();
        String searchValue =
txtSearchValue.getText().trim().toLowerCase();

        if (searchValue.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Please enter a search
value.", "Warning", JOptionPane.WARNING_MESSAGE);
            return;
        }

        StringBuilder results = new StringBuilder();
        int matchCount = 0;

        for (BunItem item : bunList) {
            boolean isMatch = false;

            switch (criteria) {
                case "Product ID":
                    isMatch =
item.getProductId().toLowerCase().contains(searchValue);
```

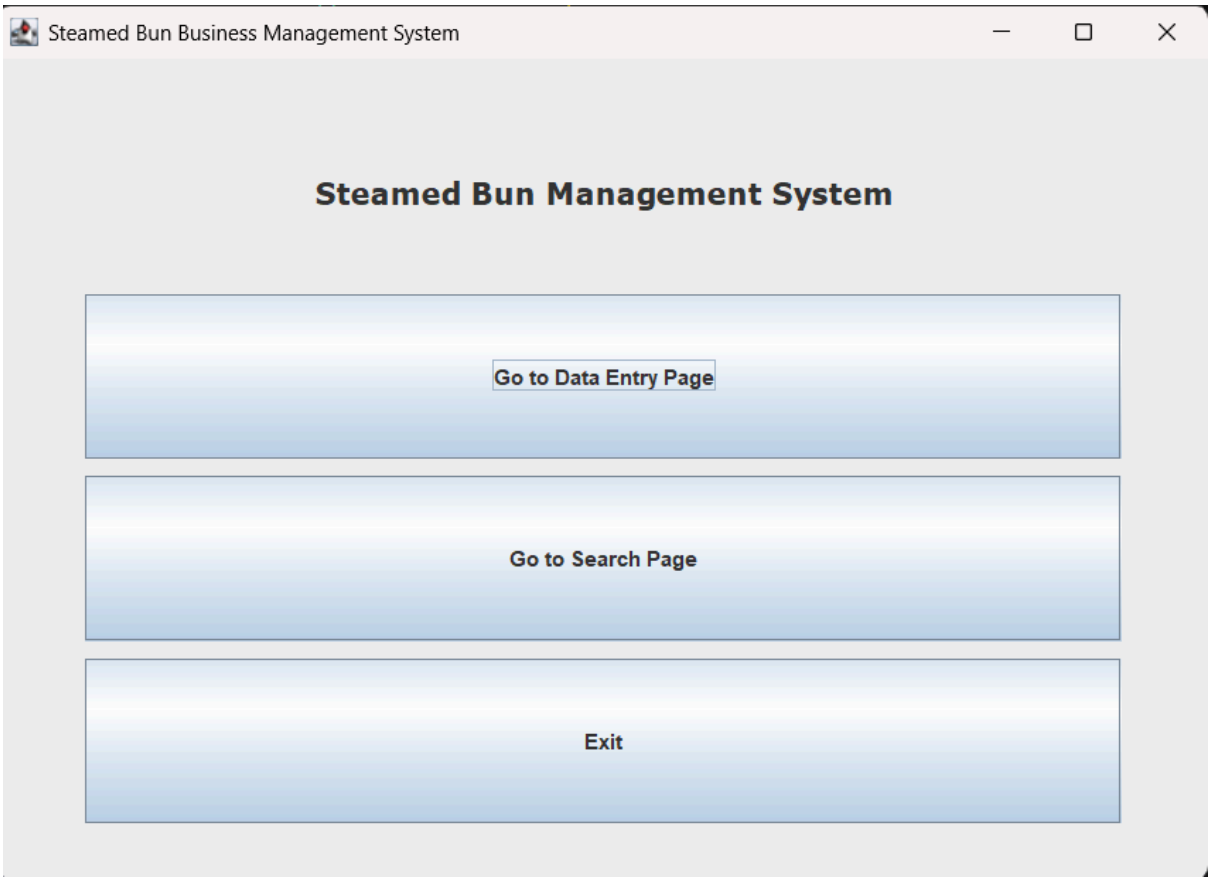
```
                    break;
                case "Product Name":
                    isMatch =
item.getName().toLowerCase().contains(searchValue);
                    break;
                case "Price":
                    try {
                        double targetPrice =
Double.parseDouble(searchValue);
                        isMatch = (item.getPrice() == targetPrice);
                    } catch (NumberFormatException ignored) {}
                    break;
                case "Quantity":
                    try {
                        int targetQty = Integer.parseInt(searchValue);
                        isMatch = (item.getQuantity() == targetQty);
                    } catch (NumberFormatException ignored) {}
                    break;
                case "Spice Level":
                    // Safe subclass check (Polymorphism & Type
Checking)
                    if (item instanceof SavoryBun) {
                        SavoryBun savory = (SavoryBun) item;
                        isMatch =
savory.getSpiceLevel().toLowerCase().contains(searchValue);
                    }
                    break;
                case "Sweetness Level":
                    // Safe subclass check (Polymorphism & Type
Checking)
                    if (item instanceof SweetBun) {
                        SweetBun sweet = (SweetBun) item;
                        isMatch =
sweet.getSweetnessLevel().toLowerCase().contains(searchValue);
                    }
                    break;
            }

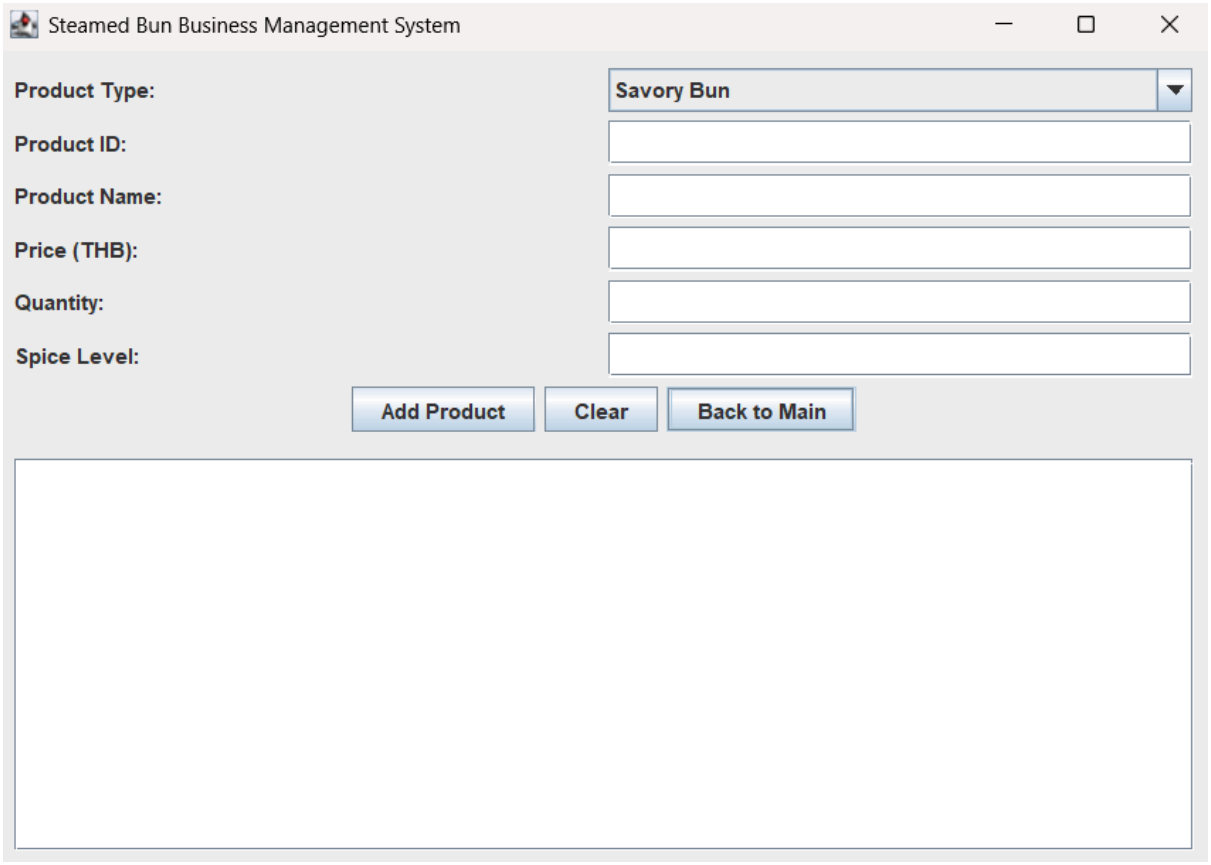
            if (isMatch) {
                results.append(item.toString()).append("\n");
                matchCount++;
            }
        }
```

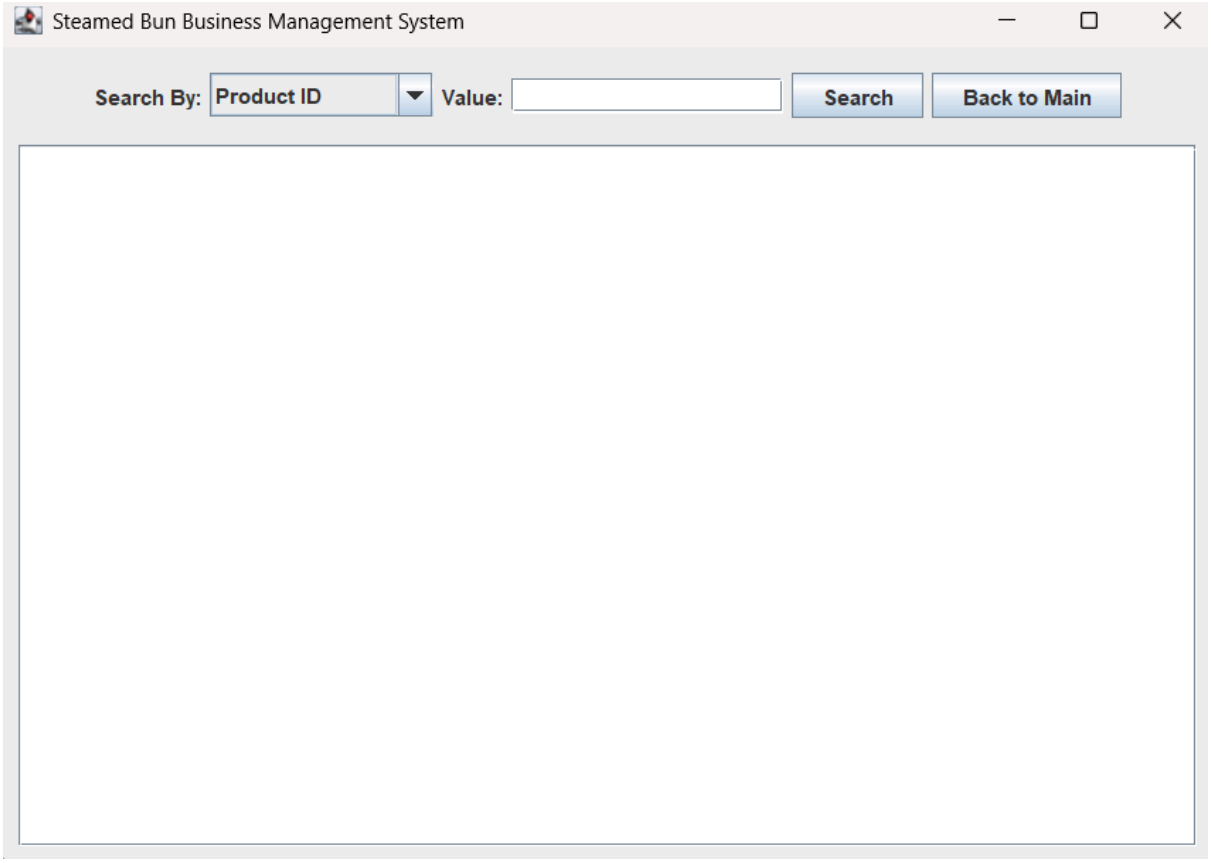
```
        if (matchCount == 0) {
            txtAreaSearchResults.setText("No matching records found.");
        } else {
            txtAreaSearchResults.setText(results.toString());
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            new SteamedBunApp().setVisible(true);
        });
    }
}
```

Java Swing GUI




Steamed Bun Business Management System
Product Type:
Savory Bun
Product ID:
Product Name:
Price (THB):
Quantity:
Spice Level:
Add Product
Clear
Back to Main


Steamed Bun Business Management System
Search By:
Product ID
Value:
Search
Back to Main